that the machine-specific part of the translation is minimized, making it easier to port the languages to different operating systems. (In fact, YARV internally compiles to a byte code at run time before interpreting it, but this byte code version is typically for internal use only.)

Another way of blurring the distinction is by using a Just-In Time compiler (JIT). A JIT compiler operates at run time and starts as an interpreter, but can then compile frequently used code as it executes from Ruby's byte code to machine code. Typically, this increases initial startup cost because of the extra compilation step, but if parts of the code are executed repeatedly in a long-lived process, the machine-compiled versions will eventually be faster.

Ruby ships with two different JIT implementations: MJIT and YJIT.

MJIT

MJIT—the "m" stands for "method-based"— is the older of the two JIT compilers. You can enable MJIT on your Ruby execution by using --mjit when you call Ruby:

```
$ ruby options.foo --mjit
```

MJIT will watch for methods that are called frequently in your code and will compile methods that it sees as having high-use. That's where the "method-based" name comes from, the compiler works at the method level. MJIT's logic is written in Ruby, and it is basically a wrapper around the native GCC compiler, calling out to compile frequently used methods and managing them so that the compiled versions are used in the future.

From your perspective as a Ruby developer, MJIT should run just like YARV, but with some kind of performance boost. Exactly how much of a boost depends on your application parameters (and as this is written, there's not a lot of solid Ruby 3.2 benchmarks to choose from), but one set of benchmarks had MJIT about 20% faster than Ruby 3.1.[1] (For clarity, these benchmarks come from the TruffleRuby team, but it's also as comprehensive a comparison as we've seen.)

A handful of command-line options change how MJIT works; however, most of these are for testing purposes if you were actually working on MJIT, rather than just writing Ruby:

--mjit-call-threshold=num	Notes the number of calls to a method needed to trigger MJIT compilation. The default is 10,000. You might lower this if you were testing MJIT itself.
--mjit-debug	Enables debugging mode, which is slow and something you'd do only if you were working on MJIT itself.
--mjit-max-cache=num	Changes the maximum number of compiled methods MJIT stores in a cache. The default is 100.
--mjit-save-temps	Causes temporary files to be saved to 'tmp' or to the '$TMP' environment variable if it exists.
--mjit-verbose=num	Logs errors of the given level priority or lower to the standard error stream. The default is 0, meaning critical errors only.
--mjit-wait	If set, causes the code to wait until the MJIT compilation finishes. This is another option for testing MJIT itself.
--mjit-warnings	If enabled, prints MJIT warnings to the standard output stream.

1. https://eregon.me/blog/2022/01/06/benchmarking-cruby-mjit-yjit-jruby-truffleruby.html